Ex.9 : Hamming Code

HammingServer.java

```
import java.io.*;
import java.net.*;

public class HammingServer {
   public static void main(String[] args) throws Exception {

      ServerSocket ss = new ServerSocket(5000);
      System.out.println("Server waiting...");
      Socket s = ss.accept();

      DataInputStream din = new DataInputStream(s.getInputStream());
      DataOutputStream dout = new DataOutputStream(s.getOutputStream());

      String data = din.readUTF(); // 7-bit dataword
      System.out.println("Dataword received: " + data);

      int[] h = new int[12]; // 1 to 11

      // Place data bits
      h[3]  = data.charAt(0) - '0';
      h[5]  = data.charAt(1) - '0';
      h[6]  = data.charAt(2) - '0';
      h[7]  = data.charAt(3) - '0';
      h[9]  = data.charAt(4) - '0';
      h[10] = data.charAt(5) - '0';
      h[11] = data.charAt(6) - '0';

      // Parity bits (Even parity – Forouzan)
      h[1] = h[3] ^ h[5] ^ h[7] ^ h[9] ^ h[11];
      h[2] = h[3] ^ h[6] ^ h[7] ^ h[10] ^ h[11];
      h[4] = h[5] ^ h[6] ^ h[7];
      h[8] = h[9] ^ h[10] ^ h[11];

      String codeword = "";
      for (int i = 1; i <= 11; i++)
         codeword += h[i];

      System.out.println("Codeword generated: " + codeword);
      dout.writeUTF(codeword);

      s.close();
      ss.close();
   }
}
         System.out.print("Corrected Codeword: ");
         for (int i = 1; i <= 11; i++)
            System.out.print(h[i]);
         System.out.println();
      }

      s.close();
   }
}
```

HammingClient.java

```
import java.io.*;
import java.net.*;
import java.util.*;

public class HammingClient {
   public static void main(String[] args) throws Exception {

      Socket s = new Socket("localhost", 5000);
      DataOutputStream dout = new DataOutputStream(s.getOutputStream());
      DataInputStream din = new DataInputStream(s.getInputStream());
      Scanner sc = new Scanner(System.in);

      System.out.print("Enter 7-bit dataword: ");
      String data = sc.next();
      dout.writeUTF(data);

      String code = din.readUTF();
      System.out.println("Codeword received: " + code);

      int[] h = new int[12];
      for (int i = 1; i <= 11; i++)
         h[i] = code.charAt(i - 1) - '0';

      // Introduce error (demo)
      h[6] = h[6] ^ 1;
      System.out.print("Codeword after error insertion: ");
      for (int i = 1; i <= 11; i++)
         System.out.print(h[i]);
      System.out.println();

      // Syndrome calculation
      int p1 = h[1] ^ h[3] ^ h[5] ^ h[7] ^ h[9] ^ h[11];
      int p2 = h[2] ^ h[3] ^ h[6] ^ h[7] ^ h[10] ^ h[11];
      int p4 = h[4] ^ h[5] ^ h[6] ^ h[7];
      int p8 = h[8] ^ h[9] ^ h[10] ^ h[11];

      int errorPos = p1 * 1 + p2 * 2 + p4 * 4 + p8 * 8;

      if (errorPos == 0) {
         System.out.println("Integrity Check: NO ERROR");
      } else {
         System.out.println("Integrity Check: ERROR at position " + errorPos);
         h[errorPos] = h[errorPos] ^ 1; // correction

         System.out.print("Corrected Codeword: ");
         for (int i = 1; i <= 11; i++)
            System.out.print(h[i]);
         System.out.println();
      }

      s.close();
   }
}
```

Server Output:

```
Server waiting...
Dataword received: 1010111
Codeword generated: 10110101111
```

Client Output:

```
Enter 7-bit dataword: 1010111
Codeword received: 10110101111
Codeword after error insertion: 10110001111
Integrity Check: ERROR at position 6
Corrected Codeword: 10110101111
```